WHO DECIDES? WHO ACTS?

Agentic planning in practice

Four agents, one coding task, and the division of work inside the agent loop.

A planner decides what to do next: what to inspect, how to split the work, which action to try, how to respond to feedback, and when to finish. In these runs, those choices are visible through the model's messages and tool calls.

THE SHARED TASK

Repair **agentstats**: correct cached-token counting, count repeated tasks and calls once, add grouped reports and CSV export, preserve the existing command, and add tests.

MODEL

Chooses the next action

Interprets the task, proposes code and tests, selects tools, and evaluates feedback.

TOOLS

Perform concrete operations

Read and write files; execute Python, shell commands, tests, and calculations.

HARNESS / RUNTIME

Runs the interaction loop

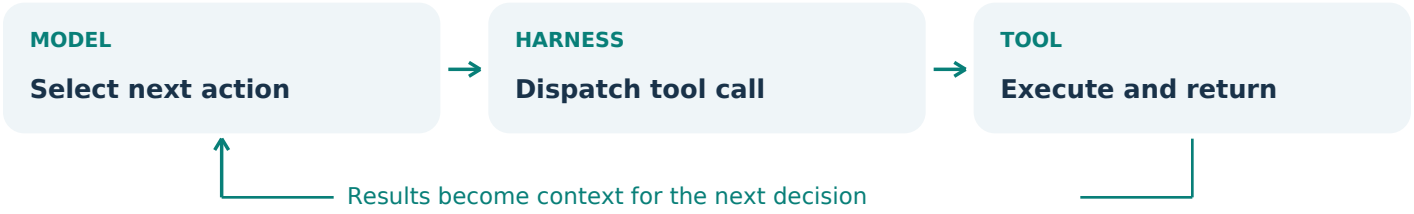
Supplies context and tools, handles permissions, dispatches calls, and returns results.

TASK / TODO STATE

Stores a visible checklist

Qoder and DSH used it. The model writes the items and statuses; the list does not choose actions.

HOW THE ROLES WORK TOGETHER



External means outside model generation. Local file tools and Python execution count; internet access is not required. The private reasoning behind a choice is not fully visible in these logs.

Final check: After the agent's own checks and completion claim, the researcher's evaluator grades the code. Each of the four saved results passed 18/18 checks for this one task.

WORKFLOW 01 / STAGES EXPRESSED IN CONVERSATION

Claude Code

The visible sequence includes a check of whether the tests themselves were useful.

`Model: claude-opus-5`

CLI 2.1.267 | Final agent tests: 25 passed | Independent acceptance: 18/18

01 MODEL + READ / BASH

Inspect the project

The model requests the task, source, tests, and sample data; tools return their contents.

02 MODEL + BASH

Establish the starting failures

The model runs the original suite. The terminal reports 9 tests, with 3 failures.

03 MODEL + BASH

Repair the implementation

The model drafts edits; shell/Python commands write the files. All 9 original tests pass.

04 MODEL + BASH

Recompute the sample totals

A separate script recalculates expected counts and tokens from the sample input.

05 FEEDBACK + MODEL + BASH

Repair the new tests

A syntax error is fixed. A later 64-test pass reveals repeated inherited tests; this becomes 25.

06 MODEL + BASH

Review and finish

The model checks edge cases and documentation, reruns 25 tests, then reports completion.

PLAN REPRESENTATION

Stages in the conversation

No Task/Todo call appears. The ordering is expressed in progress messages and actions.

OBSERVED FEEDBACK

A green test result was not the end

After 64 tests passed, the model identified duplicated test execution and reorganized the test classes. The final suite had 25 distinct tests.

WHO DID THE WORK?

Model directs; tools execute

Read retrieved files. Bash ran all file-changing scripts, calculations, and test commands.

OBSERVED TOOL USE

Read 8 / Bash 20

No task-list tool was called.

Evidence [C]: raw lines 71-98, 103-135, 156-163. Test outputs confirm 25 distinct tests; the final prose contains an inconsistent new-test count.

WORKFLOW 02 / BATCH INSPECTION AND TWO PATCHES

Codex

Independent operations were sometimes grouped into concurrent tool batches.

`Model: gpt-6-astra / reasoning effort: max`

CLI 0.154.0 / exec | Final agent tests: 29 passed | Independent acceptance: 18/18

01 MODEL + EXEC_COMMAND

Collect the relevant context

Read the task, project instructions, source, tests, and samples; batch some independent reads.

02 MODEL + EXEC_COMMAND

Confirm the original bugs

The original suite returns 9 tests with 3 failures. The model identifies the counting defects.

03 MODEL + APPLY_PATCH

Apply the first set of changes

Patch record handling, reports, CLI behavior, and README to implement the requested behavior.

04 MODEL + APPLY_PATCH

Add contract-focused checks

A second patch adds tests and a report adjustment, including boundaries and CSV consistency.

05 MODEL + PARALLEL TOOLS

Verify the modified project

The full suite passes 29 tests. Another command checks the diff and sample-file integrity.

06 MODEL + EXEC_COMMAND

Check the interface and finish

Inspect final status, diff, and CLI help; the model then reports that the task is complete.

PLAN REPRESENTATION

Stages in the conversation

No dedicated plan-update tool was called. Stage choices are visible in messages and tool requests.

ACTION ORGANIZATION

Batching does not mean more agents

Eight exec orchestration calls contained 11 command calls and 2 patch calls. Four batches used Promise.allSettled. No sub-agent delegation was observed.

VERIFICATION ROUTE

Expected behavior in tests

Validation used fixed expected values, edge cases, and output consistency tests. No separate sample-recount script was observed.

OBSERVED RECOVERY

29 tests passed after edits

No new test failure drove another implementation repair.

Evidence [X]: raw lines 13-87; export lines 19-33. Two earlier CLI/model compatibility failures are excluded from this successful coding trajectory.

WORKFLOW 03 / FOUR EXPLICIT TASKS

Qoder

The native log reveals planning-tool calls that the plain-text export omitted.

`Model: qmodel_38max / UI label: Qwen3.8-Max`

CLI 1.1.53 | Final agent tests: 48 passed | Independent acceptance: 18/18

01 MODEL + READ / BASH

Inspect code and samples

The model reads the requirements and current implementation before creating its task list.

02 MODEL + TASKCREATE

Save four work items

Counting/deduplication; grouping; CSV/CLI behavior; tests, independent checks, and documentation.

03 MODEL + WRITE / EDIT / BASH

Implement, then compare the baseline

The modified code passes 9 tests. A temporary copy of the original then confirms 3 failures.

04 FEEDBACK + MODEL + TOOLS

Correct the verification itself

A check wrongly expects a model name in overall output. The model fixes the test to use grouping.

05 MODEL + EDIT / BASH

Review semantics after tests pass

After 48 tests pass, the model fixes read_records diagnostics and shares the task-identity helper.

06 MODEL + TOOLS + TASKUPDATE

Complete final checks and close tasks

Rerun the 48-test suite, check outputs and documentation, then mark the last task complete.

PLAN REPRESENTATION

4 creates / 6 updates

The model supplied the task descriptions and statuses. Task tools recorded these choices.

STATE VERSUS EXECUTION

A checked box was a coarse milestone

Tasks 1-3 were marked complete before later related implementation edits. They were not reopened. The list did not behave like a strict execution schedule.

VERIFICATION ROUTE

Recount and review

A temporary script independently recalculated sample totals. Later checks also reviewed direct function behavior.

OBSERVED TOOL USE

Read / Bash / Write / Edit

Plus TaskCreate and TaskUpdate; 52 calls in total.

Evidence [Q]: raw lines 57-71, 100-117, 153-179, 219-273. Some reads/checks were issued in batches; this was not an entirely serial trace.

WORKFLOW 04 / A CHECKLIST AND TOOL CONSTRAINTS

DeepSeek Harness

The model advanced through the modules and adapted to execution feedback.

`Model: deepseek-official / deepseek-flash / effort: high`

Standard preset | Final agent tests: 47 passed | Independent acceptance: 18/18

01 MODEL + READ / BASH

Inspect the project

Read the task, code, original tests, and sample records through file and terminal tools.

02 MODEL + TODO_WRITE

Store five work items

Records; reporting; CLI; tests and README; final verification. One starts in progress.

03 FEEDBACK + MODEL + READ / WRITE

Repair the three modules

A write is blocked until a tracked read occurs. The model reads, retries, then follows that pattern.

04 FEEDBACK + MODEL + BASH

Test and fix the recount script

The 9 original tests pass. A grouped-versus-overall comparison error is corrected in the script.

05 MODEL + TOOLS

Expand and verify behavior

Add 38 tests; all 47 pass. Extend grouping, update docs, and run the final checks.

06 MODEL + TODO_WRITE / PRESENT

Close the list and finish

Mark all five todos complete, present the changed files, and issue the final response.

PLAN REPRESENTATION

5 items / 2 list writes

The first list had 1 active and 4 pending items. The second marked all 5 complete, with no intermediate updates.

OBSERVED FEEDBACK

A tool rule changed the next action

Reading source through bash did not satisfy the tracked-read requirement. A write failed; the model then used read and retried successfully.

VERIFICATION ERROR

2 versus 26

The script compared one group with the overall total. The model fixed the comparison; no product edit occurred between the two checks.

OBSERVED TOOL USE

27 calls / 28 model steps

One tool call per step; step 28 was the final answer.

Evidence [D]: session.v3.jsonl lines 38-54, 79-105, 129-157. Cached source [S] confirms that todo_write stores state; a separate runtime drives the loop.

INTERPRETATION / BASED ON THIS TASK ONLY

What is shared, what differs?

All four combine model-generated decisions with external execution and feedback.

Agent	Visible plan state	Execution pattern	Feedback behavior
Claude	Stage messages; no task-list call	Shell-driven edits and a separate sample recount	Repairs test generation and removes duplicated test execution.
Codex	Stage messages; no plan-tool call	Batched independent tools; two patches; contract tests	Uses original failures to guide fixes; 29 tests pass after edits.
Qoder	Four task records with status updates	File tools, separate recount, and repeated verification	Corrects faulty assertions; reviews interface semantics after tests pass.
DSH	Five todos; initial and final snapshots	Serial calls in this run; separate sample recount	Adapts to the read-before-write rule and fixes its recount comparison.

INTERNAL AND EXTERNAL

Who is doing the planning?

The model generates the next action and interprets the returned result.

The tools perform file operations, computation, and tests. These are external to model generation.

The checklist stores what the model wrote. No external planning solver was observed.

LIMITS OF THIS OBSERVATION

What remains unknown?

The logs do not reveal the model's full private reasoning or all candidate plans it considered.

No separate planner/executor delegation or explicit search tree was observed.

Local corrections were visible; a redesign of the entire plan was not established.

No ranking follows from this run. Models, prompts, permissions, and tool interfaces differed. Each system completed one task and passed 18/18 external checks. Tool-call counts and self-authored test counts use different units and coverage.

AUDIT TRAIL / HOW TO VERIFY THE ACCOUNT

Evidence and reading guide

This PDF summarizes saved Stage A artifacts. No agent or evaluator was rerun to prepare it.

[C] Claude Code / native transcript

`12e21530-1258-4d5b-9dc2-bf8eb336c291.jsonl`

Key raw lines: 71, 98, 109, 125, 129, 156. Native version: 2.1.267.

[X] Codex / native transcript and successful export

`rollout-2026-09-15T20-24-49-01a0a79a-1a1a-7d90-9cba-4afae6c01d38.jsonl`

Raw: 13-87. Export codex-A-retry2.jsonl: 19-33. Version 0.154.0; effort max.

[Q] Qoder / native transcript

`d2178ce7-b523-48f7-8c8b-a5dfed9fc24e.jsonl`

Key raw lines: 57-71, 100-117, 153-179, 219-273. Native version: 1.1.53.

[D] DeepSeek Harness / exported session

`dsh-session-session-acd72d86-ae85-47f6-a580-28637f2929e3.zip`

Member session.v3.jsonl: 38-54, 79-105, 129-157. Model effort: high.

[S] Implementation cross-check. In the local DSH cache, `dsh-tool-todo/lib/index.js` lines 170-183 store the model-supplied list; `dsh-agent-loop/lib/index.js` lines 1108-1119 drive step completion; `dsh-tool-fs/lib/index.js` lines 545-549 explain the write rejection. The cache version is 0.1.5-rc.1; the session does not independently establish the CLI version.

Acceptance and provenance. Each system has a saved `records/grade-A.json` and `records/code-A.zip` under `~/planner_probe_runs/<agent>/natural_01/`. The audit verified 18/18 checks per system and exact agreement between snapshot files and their workspace counterparts.

Full local paths, call records, and source hashes:

`/Users/mingshi/Desktop/planner_probe_v1/researcher/analysis/stage_A_planner_evidence.json`

Counting rule: 18 checks describe one coding task, not 18 tasks. The sample value of 17,680 tokens belongs to the input dataset, not agent usage. Agent-written recount scripts are separate implementations, but remain distinct from the researcher's independent evaluator.